

DEFINIZIONE DI CALCOLO NUMERICO (Libro di testo pag 132)

Il **calcolo scientifico** - o **calcolo numerico** o **analisi numerica** - è la disciplina che si occupa della ricerca o dell'implementazione di algoritmi per risolvere con il computer problemi rappresentati in forma matematica.

In particolare, con il calcolo numerico si mira a trovare soluzioni approssimate per quei problemi matematici **che non sono risolvibili per via analitica**, cioè ricavando con carta e matita formule esatte per la soluzione. Esempi di questo tipo di problemi sono la ricerca di valori di certe funzioni, il calcolo di integrali definiti e la risoluzione di equazioni o sistemi di equazioni.

L'analisi numerica è una scienza sia teorica sia sperimentale: si basa infatti su assiomi e dimostrazioni di teoremi, come il resto della matematica, ma usa anche i **risultati empirici** che si ottengono testando al computer il funzionamento dei diversi possibili algoritmi ideati per risolvere un dato problema.

Tra i fondamenti del calcolo numerico, ricordiamo le proprietà fondamentali degli algoritmi. Ricordiamo innanzitutto due caratteristiche che ogni buon algoritmo deve avere:

- I passi elementari che lo compongono devono essere espressi in modo preciso, così che un calcolatore possa interpretarli senza ambiguità;
- I passi, incluse le loro eventuali iterazioni, devono essere in numero finito, cioè non illimitato, così che un calcolatore possa eseguirli in un tempo finito.

Inoltre vogliamo che l'algoritmo sia efficiente, cioè che faccia buon uso delle risorse del computer, in particolare del tempo di calcolo necessario per eseguirlo.

Parte I

Fondamenti di calcolo numerico e applicazione in semplici problemi di Ingegneria

Capitolo 1

Introduzione al calcolo numerico

La trattazione dei semplici metodi ed algoritmi di calcolo numerico presentata nel seguito in queste note non richiedono al lettore specifiche competenze teoriche sull'argomento. Tuttavia, riteniamo utile in questo capitolo introdurre alcune problematiche legate alle elaborazioni numeriche sui calcolatori. In particolare, consideriamo

- (i) le approssimazioni dovute all'aritmetica a precisione finita dei calcolatori e
- (ii) alla discretizzazione dei problemi analitici, e
- (iii) l'analisi degli errori associati a tali approssimazioni.

1.1 Rappresentazione dei numeri in un calcolatore

Parlando di numeri ed operazioni in un calcolatore, sorge l'interrogativo su quante cifre si possono utilizzare per la loro rappresentazione. Noi siamo abituati a non porci tale problema (basta aggiungere cifre!!), **ma in un calcolatore il numero di dispositivi hardware che possono contenere e manipolare le cifre è finito**. Pertanto, siamo costretti a riservare alla rappresentazione di ogni numero uno "spazio" finito di memoria costituito da una sequenza finita di *bit* (*binary digit*) o *byte* (gruppi di 8 bit).

Ogni calcolatore permette al programmatore di scegliere tra diverse convenzioni di rappresentazione o *tipi di dato*. I tipi di dato possono differire nel numero di bit utilizzati (lunghezza della parola), ma anche nel formato di rappresentazione. Si noti comunque, che qualunque sia la codifica prescelta, la rappresentazione dei numeri reali nel calcolatore è soggetta ad approssimazioni; tali approssimazioni, come vedremo, si propagano nel corso dell'esecuzione delle operazioni, causando errori numerici anche importanti.

Il calcolo numerico è la disciplina che studia le proprietà dell'esecuzione delle operazioni tramite calcolatore, consentendo di valutare l'entità degli errori numerici introdotti durante l'esecuzione delle operazioni.

Considerato il numero finito di byte disponibili per la codifica dei numeri in un calcolatore, possiamo rappresentare solo quei numeri che, in base alla convenzione di rappresentazione scelta, possono essere contenuti nello spazio previsto per ciascuno di essi; **questi numeri sono detti numeri di macchina**.

1.1.1 Numeri interi

La rappresentazione di un numero intero comporta la rappresentazione del **segno** del numero oltre al suo valore. Se n bit sono disponibili per rappresentare un intero, la rappresentazione in modulo e segno¹ utilizza il primo bit (la posizione più a sinistra) come bit di segno; per convenzione, 0 indica un numero positivo e 1 un numero negativo. Con questa convenzione, quando sono disponibili n bit come lunghezza fissata per contenere i dati, possono essere rappresentati gli interi compresi fra $-(2^{n-1})$ e $+(2^{n-1}-1)$. Infatti, **essendo un bit utilizzato per la rappresentazione del segno, solo $n - 1$ bit sono disponibili per rappresentare il modulo del numero** (v. Tab. 1.1).

Tipo	Allocazione in memoria	Accuratezza	Insieme di rappresentazione
signed int	2 4	-	$\{-2^{15}, \dots, 2^{15}-1\}$
unsigned int	2 4	-	$\{0, 2^{15}-1\}$
float	4	6 cifre dec.	$\pm\{10^{-38}, 10^{38}\} \cup \{0\}$
double	8	15 cifre dec.	$\pm\{10^{-308}, 10^{308}\} \cup \{0\}$

Tabella 1.1: I valori riportati **sono solo indicativi** e possono variare da macchina a macchina. Compatibilmente con il numero di bit (cifre) disponibili, **la rappresentazione di un numero intero di un calcolatore risulta esatta**. Anche l'aritmetica sui numeri interi risulta esatta osservando tuttavia che (i) il risultato di una divisione tra interi produce ancora un intero, con conseguente perdita della parte frazionaria del risultato (del resto della divisione) e che (ii) il risultato può essere al di fuori dell'intervallo di rappresentazione.

Ad esempio, con numeri a 8 bit, si potrebbe verificare un inconveniente come quello qui evidenziato:

$$\begin{array}{r}
 0000\ 0001\ + \\
 1111\ 1111\ = \\
 \hline
 1\ 0000\ 0000
 \end{array}
 \qquad
 \begin{array}{r}
 1\ + \\
 255\ = \\
 0
 \end{array}$$

in cui la prima cifra, che è la più significativa, viene inesorabilmente persa portando a un risultato manifestamente errato. Tale situazione prende il nome di **overflow**.

Per comprendere le proprietà dell'aritmetica finita è utile far ricorso ad una rappresentazione grafica degli interi analoga a quella, basata su una retta, utilizzata per l'aritmetica comune. Sulla retta di Fig. 1.1 sono rappresentati gli interi: un'operazione di somma tra numeri positivi consiste nello spostarsi su tale retta, verso destra a partire dal primo addendo, di un segmento di lunghezza pari al valore del secondo addendo. Lavorando in aritmetica finita, se si supera il limite della rappresentazione si ritorna al valore iniziale, come nell'esempio precedente.

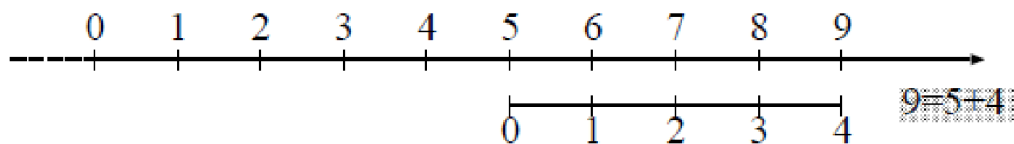


Figura 1.1: La retta dei numeri interi

Quindi, la rappresentazione non sarà più una retta, bensì una circonferenza. La Fig. 1.2 illustra la rappresentazione binaria su 3 bit. In essa, uno spostamento di un segmento di lunghezza 4 a partire da 5 (101) porterebbe ad attraversare lo zero, arrivando a 1. Infatti il risultato corretto, 9, non è rappresentabile su 3 bit e l'operazione dà luogo a overflow.

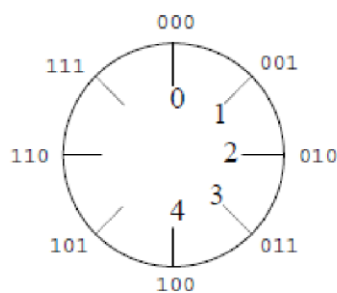


Figura 1.2: Rappresentazione degli interi in aritmetica finita

¹Esistono diverse altre codifiche binarie di numeri interi. Tra queste, una delle più utilizzate per la semplificazione dell'esecuzione di operazioni aritmetiche tra numeri interi, è la **rappresentazione in complemento a 2**.

1.1.2 Numeri reali

In generale, un numero reale può essere rappresentato solo in forma approssimata con arbitraria precisione mediante allineamenti finiti di cifre. Dovendo utilizzare un calcolatore digitale per calcoli

scientifici di massima precisione, con valori molto grandi o molto piccoli, il principale problema da affrontare è la limitatezza dello spazio (cioè il numero di bit) a disposizione per rappresentare i numeri. Naturalmente, la strada da seguire non è quella di aumentare arbitrariamente il numero di bit impiegati per rappresentare ciascun numero. Si ricorre invece a un particolare tipo di notazione denominata in **virgola mobile (floating point)** che utilizza la notazione esponenziale per la codifica dei numeri reali; per questa rappresentazione non esiste un unico standard, perciò consideriamo una delle tante possibili convenzioni. Secondo tale rappresentazione, ogni numero reale x può essere scritto nella forma:

$$x = mB^e$$

dove m è un numero reale, detto *mantissa*, B è la *base* del sistema di numerazione scelto ($B = 10$ per il sistema decimale, $B = 2$ per il sistema binario), ed e è un *esponente* intero positivo, negativo o nullo. Questa rappresentazione non è unica; infatti abbiamo

$$x = mB^e = m'B^{e-1} = m''B^{e+1} = \dots, \quad m' = mB, \quad m'' = m/B.$$

Si noti che l'insieme dei numeri generabili con questa notazione è un sottoinsieme dei numeri reali, distribuito in modo non uniforme; precisamente, vi saranno valori estremamente vicini tra loro nell'intorno del numero 0 ed estremamente lontani fra loro nell'intorno del numero massimo esprimibile, positivo o negativo.

La notazione esponenziale consente di scrivere in poco spazio numeri molto grandi o molto piccoli evitando di sprecare un numero elevato di bit con valori nulli atti soltanto ad indicare l'ordine di grandezza del numero. Aumentando così il numero di bit significativi, aumenta la *precisione* del numero. In pratica, nei calcolatori digitali i byte destinati a memorizzare un numero reale sono così suddivisi:

- un bit per il segno della mantissa (e quindi dell'intero numero);
- t bit per la mantissa ($m_i < m < m_s$);
- q bit per l'esponente ($M_i < e < M_s$).

La rappresentazione di $x \neq 0$ si dice normalizzata quando

$$B^{-1} \leq |m| < 1 \quad (\text{non nella convenzione 754} \rightarrow 1.\text{mantissa}).$$

Fissata la base B , ogni numero reale $x \neq 0$ è univocamente definito dalla coppia (m, e) , per cui è sufficiente memorizzare quest'ultima. Non esistendo la rappresentazione normalizzata del numero $x = 0$, per convenzione si individua tale numero con la coppia $(0, 0)$.

Le operazioni su numeri reali rappresentati con un numero finito di cifre possono dare luogo a fenomeni di *overflow* (numeri con esponente $e > M_s$) o di *underflow* (numeri con esponente $e < M_i$).

1.2 Approssimazioni ed errori

1.2.1 Problemi numerici e algoritmi

Per problema numerico intendiamo una descrizione chiara e non ambigua di una relazione funzionale tra i dati (*input*) del problema (che costituiscono le variabili indipendenti) e i risultati desiderati (*output*) (v. Fig. 1.3).

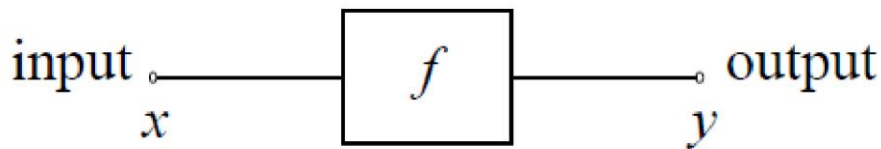


Figura 1.3: Schematica rappresentazione di un problema numerico

I dati x e i risultati y devono essere rappresentabili da vettori (di dimensione finita) di numeri. La relazione funzionale f può essere espressa sia in forma esplicita $y = f(x)$, sia implicita $f(x, y) = 0$.

Per algoritmo di un problema numerico intendiamo una descrizione completa e ben definita di operazioni che permetta di trasformare (in un numero finito di passi) ogni vettore di dati (permissibili) x nel corrispondente output y^* , non necessariamente uguale a y . Ad ogni problema numerico possiamo associare più algoritmi, ognuno dei quali in genere fornirà risultati con precisione diversa.

Il problema numerico è caratterizzato dalla sola relazione funzionale esistente tra input e output; in un algoritmo invece, ogni singolo passo (operazione) deve essere definito chiaramente ed inequivocabilmente, dai dati sino alla determinazione effettiva dei risultati.

Nei capitoli che seguono parleremo di problemi e di algoritmi; in particolare **esamineremo algoritmi per il calcolo della soluzione di problemi numerici spesso ottenuti approssimando (o meglio, discretizzando) modelli matematici.**

Generalmente, tra la soluzione analitica e quella numerica, si osserva una discrepanza, o errore, dovuta al fatto che le tecniche numeriche presuppongono un'approssimazione. Errori di questo tipo sono caratteristici della maggior parte delle tecniche descritte in queste note. Ciò può sembrare a prima vista contrario rispetto a quanto ci si aspetta da una tecnica ingegneristica valida. Gli studenti e i consulenti di ingegneria, per esempio, tentano costantemente di limitare gli errori nel proprio lavoro: quando lo studente sostiene esami o esegue esercizi, viene penalizzato per gli errori commessi. Nella pratica professionale, poi, gli errori possono costare parecchio e possono risultare addirittura catastrofici: il cedimento di una struttura o di una macchina, ad esempio, può comportare la perdita di vite umane.

Sebbene la perfezione sia un traguardo a cui mirare, essa, in pratica, non viene mai raggiunta. I metodi numerici possono introdurre simili discrepanze nell'analisi; **anche in questo caso la domanda è quale sia l'errore tollerabile.** Nel seguito sono trattati gli argomenti fondamentali relativi all'identificazione, quantificazione e minimizzazione di tali errori. In particolare verranno descritti i due tipi principali di errori: **di arrotondamento** e **di troncamento**.

Gli errori di arrotondamento sono dovuti al fatto che i calcolatori dispongono di un numero finito di cifre per rappresentare i dati.

L'errore di troncamento è la discrepanza introdotta dal fatto che i metodi numerici utilizzano delle approssimazioni per eseguire calcoli matematici e rappresentare quantità esatte. Per esempio, moltissimi **metodi numerici richiederebbero l'esecuzione di una sequenza infinita di passi di calcolo: essi sono perciò "troncati" dopo un numero di passi finito, quando si è raggiunta un'approssimazione sufficiente dei risultati.**

1.2.2 Definizione degli errori

Gli *errori numerici* sono dovuti all'uso di approssimazioni nella rappresentazione di quantità esatte e nell'esecuzione di calcoli matematici. **Gli errori di troncamento** dipendono dal fatto che una certa

procedura matematica viene approssimata da una serie di operazioni aritmetiche più semplici, mentre gli **errori di arrotondamento** risultano dall'uso di un numero limitato di cifre significative per la rappresentazione dei numeri. In entrambi i casi, la relazione tra il risultato esatto, o vero, e l'approssimazione può essere così formulata:

$$\text{valore vero} = \text{approssimazione} + \text{errore}$$

da cui troviamo che l'errore numerico è uguale alla differenza tra il valore vero e l'approssimazione:

$$e = \text{valore vero} - \text{approssimazione} \quad (1.1)$$

dove e indica l'**errore assoluto (in valore assoluto)**.

Questa definizione presenta uno svantaggio: non tiene in nessun conto l'ordine di grandezza del valore che si sta esaminando. Per esempio, un errore di un centimetro è molto più significativo quando si misura la lunghezza di un chiodo piuttosto che quella di un ponte.

Per tener conto dell'ordine di grandezza delle quantità in esame si può normalizzare l'errore rispetto al valore vero ottenendo l'*errore relativo*:

$$\epsilon = \frac{\text{errore}}{\text{valore vero}} = \frac{e}{\text{valore vero}}$$

L'errore relativo può anche essere moltiplicato per 100 in modo da esprimerlo come frazione di 100 e non di 1:

$$\epsilon_p = \frac{\text{errore}}{\text{valore vero}} 100\%$$

dove ϵ_p viene definito *errore relativo percentuale*.

Nel caso dei metodi numerici, il valore vero è noto solo quando si ha a che fare con funzioni che possono essere espresse analiticamente, come nello studio teorico del comportamento di una particolare tecnica.

Nelle applicazioni reali, invece, il risultato esatto non è mai noto a priori: in questi casi, l'alternativa è normalizzare l'errore assoluto usando la migliore stima disponibile del valore vero; normalizzare, cioè, rispetto all'approssimazione stessa

$$\epsilon_a = \frac{\text{valore vero} - \text{approssimazione}}{\text{approssimazione}} \quad (1.2)$$

dove il pedice a significa che l'errore è normalizzato rispetto a un valore approssimato. Nelle applicazioni reali, inoltre, la (1.1) non può essere usata per calcolare il termine relativo all'errore contenuto nella (1.2): una delle difficoltà che si incontrano nell'applicazione dei metodi numerici sta nella determinazione di stime dell'errore in mancanza di nozioni circa il valore vero.

Certi metodi numerici, per esempio, arrivano a un risultato procedendo per iterazioni; secondo questo tipo di approccio, l'approssimazione più recente viene calcolata sulla base della precedente. Questo processo viene ripetuto, o iterato, per calcolare approssimazioni sempre migliori. In questi

casi, la stima dell'errore che viene assunta è semplicemente la differenza tra il risultato dell'ultima iterazione e quello della precedente.

L'errore relativo, quindi, viene determinato secondo la formula

$$\epsilon_a = \frac{\text{approssimazione attuale} - \text{approssimazione precedente}}{\text{approssimazione attuale}} \quad (1.3)$$

I valori che si ottengono dalle espressioni (1.1)-(1.3) possono essere sia positivi che negativi; se l'approssimazione è maggiore del valore vero (o la penultima approssimazione è maggiore dell'ultima), l'errore è negativo; in caso contrario, l'errore è positivo. Inoltre per le espressioni dell'errore relativo, il denominatore può essere minore di zero e anche questa condizione può portare a un errore negativo. Spesso, quando si eseguono dei calcoli il segno dell'errore può non essere interessante, ma ciò che realmente importa è che il **valore assoluto dell'errore sia minore di una tolleranza Σ_s specificata in partenza.**

Pertanto, è spesso utile fare riferimento **al valore assoluto delle espressioni (1.1)-(1.3)**; in tali casi, il calcolo viene proseguito finché si ha: $|\Sigma_a| < \Sigma_s$

Errore Assoluto (in valore assoluto)

$$e = \text{valore vero} - \text{approssimazione} \quad (1.1)$$

Errore relativo (in valore assoluto)

$$\epsilon_a = \frac{\text{approssimazione attuale} - \text{approssimazione precedente}}{\text{approssimazione attuale}} \quad (1.3)$$

Se questa condizione è verificata, si assume che il risultato sia corretto entro la tolleranza Σ_s prefissata.

Corsi di Laurea in
Ingegneria Biomedica e Ingegneria Elettronica

Note di Fondamenti di Informatica I (II semestre)

Silvio Sabatini

Calcolo della radice quadrata con il METODO BABILONESE

Note storiche

I primi ad occuparsi del problema dell'estrazione della radice quadrata di un numero sono stati i babilonesi. Essi, tra i primi ad utilizzare un sistema di numerazione posizionale, avevano elaborato un procedimento per l'estrazione della radice quadrata. I babilonesi avevano ricavato la radice di 2 come 1,414222, con un errore di circa 0,000008 dal valore vero.

Applicazione

Il calcolo della radice quadrata con il metodo babilonese è concettualmente semplice.

Per calcolare la radice quadrata di un qualsiasi numero n maggiore di 0, si definisce una stima qualunque s_0 , alla quale generalmente si assegna valore 1 o $n/2$. La seconda stima s_1 sarà la media aritmetica tra s_0 e n/s_0 , e così via.

Il metodo converge molto velocemente: ciò significa che si giunge al risultato desiderato, ovvero al valore teorico (reale) della radice di n , attraverso un ridotto numero di passaggi.

L'algoritmo ottenuto può essere rappresentato da una relazione di ricorrenza:

$$\begin{cases} s_0 = \frac{n}{2} \\ s_x = \frac{s_{x-1} + \frac{n}{s_{x-1}}}{2} \end{cases}$$

dalla quale si ricava:

$$\lim_{x \rightarrow \infty} s_x = \sqrt{n}$$

infatti, più procedo nei calcoli, più mi avvicino al valore esatto della radice di n .

▪ esempio: calcolo della radice quadrata di 2:

$$s_0 = 1 \text{ oppure } 2/2=1$$

$$s_1 = (1+2/1)/2 = 3/2 = 1,5$$

$$s_2 = (1,5+2/1,5)/2 = 1,41666...$$

$$s_3 = (1,41666+2/1,41666)/2 = 1,41422...$$

▪ esempio: calcolo della radice quadrata di 3:

$$s_0 = 3/2=1,5$$

$$s_1 = (1,5+3/1,5)/2 = 1,75$$

$$s_2 = (1,75+3/1,75)/2 = 1,73214286...$$

$$s_3 = (1,73214286+3/1,73214286)/2 = 1,732050...$$

in questo caso, risulta vantaggioso assegnare ad s_0 valore $n/2$, ovvero 1,5: in questo modo viene infatti ridotto ulteriormente il numero dei passaggi svolti per giungere al risultato corretto.

Il motivo per cui il metodo funziona è semplice. Per definizione, se s_x è una stima per difetto di $\sqrt{n}$, allora n/s_x (con $s_x < \sqrt{n}$) sarà una stima per eccesso, e viceversa.

Il metodo converge quadraticamente, fornendo il numero corretto delle cifre che, in genere, raddoppia ad ogni iterazione.

È vero che la media tra i due valori non è necessariamente una stima migliore di quella iniziale.

▪ esempio: calcolo della radice quadrata di 1000:

```
s0 = 1
s1 = (1+1000/1)/2 = 500,5
s2 = (500,5+1000/500,5)/2 = 251,24900...
s3 = (251,24900+1000/251,24900)/2 = 127,614557...
s4 = (127,614557+1000/127,614557)/2 = 67,7253...
...
sx = (sx-1 + 1000/sx-1)/2 = 31,622776...
```

Ma questo può capitare solo al primo passo. In seguito, se il numero di partenza è maggiore di 1, tutte le stime successive saranno per eccesso.

Questo enunciato è dimostrabile attraverso la disuguaglianza aritmo-geometrica, la quale afferma che, dati due numeri positivi, la loro media aritmetica è sempre maggiore o uguale della media geometrica, e l'uguaglianza si ottiene solo quando i due numeri di partenza sono uguali.

definizione di media geometrica:

la media geometrica di due o più valori numerici è la radice n-esima del prodotto dei valori considerati, con n il numero dei valori.

Nel caso del calcolo della radice con il metodo babilonese, la media geometrica tra i due valori coincide sempre con il valore reale della radice di n .

Si può quindi scrivere:

$$\frac{x+y}{2} \geq \sqrt{xy}$$

▪ esempio: prendendo in considerazione la seconda stima nel calcolo della radice di 1000, la disequazione si traduce in:

$$\frac{500,5 + \frac{1000}{500,5}}{2} \geq \sqrt{500,5 \times \frac{1000}{500,5}} \rightarrow 251,24900 \geq \sqrt{1000} \rightarrow 251,24900 \geq 31,622776$$

Programmi

1. Radice Babilonese

```
#include <stdio.h>
#include <math.h>
double inputDouble(char mess[],int min)
{
    double n;
    do
    {
        printf("%s",mess);
        scanf("%lf",&n);
```

```

        if(n<min)
            //la radice quadrata di un numero negativo risulta impossibile nell'insieme
dei numeri reali
            printf("\nATTENZIONE! numero inserito non valido");
        }
        while(n<min);
        return n;
}

```

```

int inputInt(char mess[],int min)
{
    int pedice;
    do
    {
        printf("%s",mess);
        scanf("%d",&pedice);
        if(pedice<min)
            printf("\nATTENZIONE! numero inserito non valido");
    }
    while(pedice<min);
    return pedice;
}

```

```

double radiceBabilonese(double n,int pedice)
{
    if(n==0)
        //la radice quadrata di 0 è 0, non risulta opportuno entrare nel ciclo for
        return 0;
    double s=n/2; //stima iniziale
    int i;
    for(i=1;i<=pedice;i++) //numero di iterazioni corrispondente al valore del pedice
    {
        s=0.5*(s+n/s);
        printf("\ns%d = %.12lf",i,s);
    }
    return s;
}

```

```

main()
{
    printf("CALCOLO DELLA RADICE QUADRATA DI UN NUMERO CON ALGORITMO BABILONESE\n");
    double numero=inputDouble("\nInserisci numero reale positivo: ",0);
    int pedice;
    printf("\nLa radice calcolata con funzione di libreria sqrt e'%.12lf",sqrt(numero));
    pedice=inputInt("\n\nInserisci pedice/termine della successione/numero di iterazioni: ",0);
    printf("\nLa radice calcolata con algoritmo BABILONESE e'
%.12lf",radiceBabilonese(numero,pedice));
}

```

2. Radice Babilonese ricorsiva

```
double radiceBabiloneseRicorsiva(double n,int pedice)
{
    if(n==0)
        return 0;
    if(pedice==0) //caso di arresto
        return n/2;
    /*dichiaro una variabile r, in modo tale che la funzione ricorsiva, nel return, non venga
    richiamata due volte con argomenti uguali (cfr funzione ricorsiva per il calcolo della successione di
    Fibonacci che però richiama la funzione con argomenti diversi)*/
    double r=radiceBabiloneseRicorsiva(n,pedice-1);
    return 0.5*(r+n/r);
}
```

3. Radice Babilonese ed errore relativo

```
double radiceBabiloneseErroreRelativo(double n, double errore, int* pedice)
{
    if(n==0)
        return 0;
    double s=n/2,precedente;
    *pedice=0; //il pedice viene passato per indirizzo, in modo tale da garantire la modifica del
    suo valore anche nel main
    do
        /*viene questa volta utilizzato un do while (e non un ciclo for), in quanto non è noto a
        priori il numero di termini da calcolare in modo tale che il valore dell'errore relativo risulti inferiore
        a quello dell'errore tollerabile richiesto in precedenza*/
        {
            /*nella funzione per il calcolo dell'errore relativo, è necessario dichiarare una
            variabile in cui memorizzare il valore della stima precedente*/
            precedente=s;
            s=0.5*(precedente+n/precedente); //calcolo la stima successiva
            (*pedice)++; //il pedice viene incrementato ogni volta
            printf("\ns%d=%lf  errore=%lf",*pedice,s,fabs((s-precedente)/s));
            /*per definizione, nelle iterazioni l'errore relativo è uguale al quoziente della
            divisione tra differenza tra approssimazione attuale (s) ed approssimazione precedente
            (precedente), e approssimazione attuale (s), il tutto in valore assoluto (fabs)*/
        }
    while(fabs((s-precedente)/s)>errore);
    //rientro nel ciclo tutte le volte che l'errore relativo calcolato è maggiore del valore
    dell'errore tollerabile
    return s;
}
```

main()

```

{
printf("CALCOLO DELLA RADICE QUADRATA DI UN NUMERO CON ALGORITMO BABILONESE\n");
double numero=inputDouble("\nInserisci numero reale positivo: ",0), errore;
int pedice, esponente=inputInt("\n\nInserisci esponente: ",0);
printf("\nLa radice calcolata con funzione di libreria sqrt e' %lf",sqrt(numero));
errore=pow(10,-esponente);
printf("\nErrore: %lf\n",errore);
printf("\nLa radice calcolata con algoritmo BABILONESE con errore <= %lf e'
q%d=%lf",errore,pedice,radiceBabiloneseErroreRelativo(numero,errore,&pedice));
}

```

3. Radice Babilonese ed errore assoluto

```

double radiceBabiloneseErroreAssoluto(double n, double errore, int* pedice)
{
    if(n==0)
        return 0;
    double s=n/2;
    *pedice=0;
    do
        /*viene questa volta utilizzato un do while (e non un ciclo for), in quanto non è noto a
priori il numero di termini da calcolare in modo tale che il valore dell'errore assoluto risulti
inferiore a quello dell'errore tollerabile richiesto in precedenza*/
        {
            s=0.5*(s+n/s);
            (*pedice)++;
            printf("\ns%d=%lf  errore=%.12lf",*pedice,s,fabs(sqrt(n)-s));
            /*per definizione, nelle iterazioni l'errore assoluto è uguale alla differenza tra il
valore reale della radice (sqrt(n)) e l'approssimazione attuale (s), il tutto in valore assoluto (fabs).
non conoscendo il valore reale della radice a priori, utilizziamo la funzione di libreria sqrt(n)*/
        }
        while(fabs(sqrt(n)-s)>errore);
        //rientro nel ciclo tutte le volte che l'errore assoluto calcolato è maggiore del valore
dell'errore tollerabile
        return s;
}

```

Calcolo approssimato della radice cubica

Scrivere 4 funzioni, 3 iterative e 1 ricorsiva, per calcolare gli elementi della successione così definita:

$$a_0=1 \quad a_{n+1}=\frac{\left(a_n+\frac{b}{a_n^2}\right)}{2}$$

il cui limite è $\sqrt[3]{b}$.

```
main(){
    double b = inputDouble("inserisci radicando reale:");
    printf("\nCon funzione di libreria cbrt/con pow(n,1/3.0)=%.16f",cbrt(b));

    int pedice = inputInt("quale termine?",0);
    printf("\nfunzione iterativa - La radice cubica di %.16f=%.16f",b,radIt(b, pedice));
    printf("\nfunzione ricorsiva - La radice cubica di %.16f=%.16f",b,radRic(b, pedice));

    double errore = pow(10, -inputInt("errore 10^-",0));
    int maxn;
    printf("\nerrore assoluto - la radice cubica con calcolo errore è %.16f ",radEA(b,errore, &maxn));
    printf(" con maxn=%d", maxn);
    printf("\nerrore relativo - la radice cubica con calcolo errore è %.16f ",radER(b,errore, &maxn));
    printf(" con maxn=%d", maxn);
}
```

Calcolo radice cubica di un numero

inserisci b:	27		Valore "vero"	
			3	
termine della successione	valore		errore assoluto= VALORE ASSOLUTO (valore vero-a_n)	errore relativo=VALORE ASSOLUTO(a_n-a_{n-1})
a₀	1		2,000000	
a₁	14		11,000000	13,000000
a₂	7,068877551		4,068878	6,931122
a₃	3,804606118		0,804606	3,264271
a₄	2,83494376		0,165056	0,969662
a₅	3,097222743		0,097223	0,262279
a₆	2,955918505		0,044081	0,141304
a₇	3,023031727		0,023032	0,067113
a₈	2,988746677		0,011253	0,034285
a₉	3,005690298		0,005690	0,016944
a₁₀	2,997171		0,002829	0,008519
a₁₁	3,001418507		0,001419	0,004248
a₁₂	2,999291752		0,000708	0,002127
a₁₃	3,000354375		0,000354	0,001063
a₁₄	2,999822875		0,000177	0,000531
a₁₅	3,000088578		0,000089	0,000266
a₁₆	2,999955715		0,000044	0,000133
a₁₇	3,000022144		0,000022	0,000066
a₁₈	2,999988928		0,000011	0,000033
a₁₉	3,000005536		0,000006	0,000017
a₂₀	2,999997232		0,000003	0,000008
.....				

Calcolo di e con serie di Eulero

Lettera **e** come "esponenziale" o forse come "**Eulero**" ?

Forse semplicemente perché seconda vocale dopo la a, che egli aveva già utilizzato nei suoi appunti per altri scopi.

Eulero si dedicò anche al calcolo del suo valore, trovando le prime 18 cifre decimali di **e**, 2.718281828459045235 senza rilevare il meccanismo di calcolo utilizzato.

Inoltre egli dimostrò che

$$e = 1 + \frac{1}{1!} + \frac{1}{2!} + \frac{1}{3!} + \frac{1}{4!} + \frac{1}{5!} + \dots = \sum_{k=0}^{\infty} \frac{1}{k!}$$

ed è la formula che utilizzeremo nel nostro algoritmo per calcolare una approssimazione di **e**.

```
#include <stdio.h>
#include <math.h>
#include <stdlib.h>
main(){
printf("Valore di e con costante M_E=%.16lf", M_E);
int k = inputInt("\nMassimo valore di k: ",0);
printf("\nil valore di e con %d iterazioni e' %.16lf", k,euleroIt(k));
printf("\nil valore di e con %d iterazioni e' %.16lf", k,euleroRic(k));

double errore =pow(10, -inputInt("\n\nerrore 10^-",0));
printf("\nerrore: %.16lf",errore);
int kMax;
double valore=euleroEA(errore, &kMax);
printf("\nCON CALCOLO ERRORE ASSOLUTO<=%.16lf: il valore di e con
massimo valore di k=%d e' %.16lf",errore, kMax,valore);
printf("\nerrore ASSOLUTO commesso: %.16lf", fabs(M_E-valore));

valore=euleroER(errore, &kMax);
printf("\n\nCON CALCOLO ERRORE RELATIVO<=%.16lf: il valore di e con
massimo valore di k=%d e' %.16lf",errore, kMax,valore);
printf("\n\n");
system("PAUSE");
}
```

Le funzioni:

....

Leibniz e il calcolo del π

Note storiche

Con il simbolo π si intende la costante che esprime il rapporto tra la lunghezza di una circonferenza e la lunghezza del suo diametro:

$$\pi = C/2r$$

Il π è un numero **irrazionale**; la prima approssimazione che si può dare a π è quella ottenuta dalla frazione $22/7=3.142$, ma alla soluzione di questo problema si sono dedicati in ogni epoca molti matematici di cultura e paesi differenti. Uno tra questi è stato **Leibniz**.

Nel **1671**, James Gregory, determinò lo sviluppo in serie di potenze della funzione arcotangente:

$$\arctan(x) = x - \frac{x^3}{3} + \frac{x^5}{5} - \frac{x^7}{7} + \dots$$

e tre anni dopo Leibniz la applicò per ricavare una formula matematica:

$$\frac{\pi}{4} = 1 - \frac{1}{3} + \frac{1}{5} - \frac{1}{7} + \dots$$

ovvero **la serie di Leibniz**

$$\sum_{n=0}^{\infty} \frac{(-1)^n}{2n+1} = 1 - \frac{1}{3} + \frac{1}{5} - \frac{1}{7} + \dots + \frac{(-1)^n}{2n+1} + \dots = \frac{\pi}{4}$$

La somma infinita a segni alterni di tutti i reciproci dei numeri naturali dispari, partendo da 1, è uguale a un quarto di pigreco.

Supponiamo di calcolare $\pi/4$ con $n=5$ (sommo 6 termini):

$$1 - 1/3 - 1/5 - 1/7 + 1/9 - 1/11 = 1 - 0.333333 + 0.200000 - 0.142857 + 0.111111 - 0.090909 = 0.744 * 4 = 2.976 \dots$$

La serie di Leibniz è una serie a segni alterni (il segno è dato da $(-1)^n$); abbiamo la certezza che la serie converga, grazie al criterio di Leibniz.

Criterio di Leibniz

È un criterio di convergenza per le serie a **segni alterni**.
Supponiamo di avere una serie:

$$\sum_{n=0}^{\infty} (-1)^n * a_n$$

tale che siano soddisfatte tutte e tre le seguenti condizioni:

1. $a_n > 0 \forall n$ (cioè la serie è a segni alterni)
2. $a_n \rightarrow 0$ (altrimenti la serie non sarebbe convergente);
3. $a_n \geq a_{n+1}$ (cioè la serie è monotona decrescente);

allora la serie converge.

Calcolo di π con la serie di Leibniz con massimo valore di n noto

```
main() {
int n = inputInt("\nInserisci il massimo valore di n: ",0);

//n=0 è accettabile e pigreco viene approssimato a 4 (4*1)

printf("\nValore della costante pigreco (M_PI): \n%.16lf", M_PI); //libreria math.h

printf("\n%.16lf con algoritmo iterativo",4*leibnizIt(n));
printf("\n%.16lf con algoritmo ricorsivo",4*leibnizRic(n));
}

int inputInt(char mess[], int min){
//come parametri la stringa da stampare e il min valore di n accettabile
int n;
do {
printf("%s",mess);
scanf("%d",&n);
if(n<min)
printf("\nATTENZIONE! numero inserito non valido");
} while(n<min);
return n; }
```

Funzione iterativa con massimo valore di n noto (ciclo for)

```
double leibnizIt(int num){
    int i;
    double s=0;
    for(i=0; i<=num;i++)
        s+=pow(-1,i)/(2*i+1);
    return s;
}
```

Funzione ricorsiva con massimo valore di n noto

```
double leibnizRic(int num) {
    if (num==0)
        return 1;
    return pow(-1,num)/(2*num+1)+ leibnizRic(num-1);
}
```

Calcolo del PI greco con una certa tolleranza/errore

Se n non è noto a priori (ovvero non si sa quanti termini della serie si vuole sommare) o si vuole calcolare l'approssimazione del $\pi/4$ con una tolleranza scelta dall'utente, si può utilizzare un ciclo (while... o do...while, meglio do...while) che esegua il calcolo del $\pi/4$ fino a quando l'errore assoluto o l'errore relativo sia minore/abbia raggiunto un valore richiesto ($10^{-\dots}$).

Con il calcolo dell'errore assoluto |valoreVero-valoreCalcolato|

Prenderemo come "valoreVero" la costante M_PI.

Calcolare il valore approssimato di π con un errore assoluto $\leq 10^{-\dots}$ inserito in input dall'utente.
Stampare il valore massimo assunto da n.

```
main(){
    double e = pow(10, -inputInt("\nprecisione alla 10^-",1));
    //per calcolare l'errore/tolleranza printf("\ntolleranza=%.16lf",e);
    printf("\npi greco di libreria=%.16lf&",M_PI);
    /*il valore di maxn è il massimo valore di n e va inizializzato a -1

    int maxn;
    printf("\nPi greco con calcolo ERRORE ASSOLUTO<=%lf e' uguale a %.16lf",e,4*leibnizEA(e,
    &maxn));
    printf("con massimo valore di n=%d", maxn);
```

```

printf("\nPigreco con calcolo ERRORE RELATIVO<=%lf e' uguale a %.16lf ",e,4*leibnizER(e,
&maxn));
printf("con massimo valore di n=%d", maxn);
}

```

```

double leibnizEA(double imp, int*mn ){
    *mn=-1;           //0 perchè uso il preconditionale
    double s=0;
    do{
        (*mn)++;
        s+=(pow(-1, (*mn)))/(2*(*mn)+1);
    }while (fabs(4*s-M_PI)>imp);
    return s;
}

```

Con il calcolo dell'errore relativo $|(nuovoValore - vecchioValore) / nuovoValore|$

```

double leibnizER(double imp, int*mn){
    *mn=-1;
    double s=0, prec;
    do{
        (*mn)++; //qui altrimenti esco con un valore incrementato di 1
        prec=s;
        s+=(pow(-1, (*mn)))/(2*(*mn)+1);
    }
    while (fabs((s-prec)/(s))>imp);
    return s;
}

```

Calcolo della radice quadrata di un numero $a \geq 0$ con metodo di Newton (successione)

$$\begin{cases} x_0 = \frac{2}{a} \\ x_n = \frac{1}{2} * (3 * x_{n-1} - a * x_{n-1}^3) \end{cases}$$

con $\lim_{n \rightarrow \infty} x_n = \frac{1}{\sqrt{a}}$ quindi dovremo calcolare l'inverso dell'ennesimo termine per ottenere la radice.

a	5		
radice di a con sqrt	2,23606797749979		
x0	0,4000000000000000	radice	2,5000000000000000
x1	0,4400000000000000		2,2727272727272700
x2	0,4470400000000000		2,2369362920544000
x3	0,4472134944358400		2,2360684828204900
x4	0,4472135954999240		2,2360679774999600
x5	0,4472135954999580		2,2360679774997900
x6	0,4472135954999580		2,2360679774997900
x7	0,4472135954999580		2,2360679774997900
x8	0,4472135954999580		2,2360679774997900
x9	0,4472135954999580		2,2360679774997900
x10	0,4472135954999580		2,2360679774997900
x11	0,4472135954999580		2,2360679774997900
x12	0,4472135954999580		2,2360679774997900
x13	0,4472135954999580		2,2360679774997900

```
main()
{
double a=inputDouble("inserisci radicando", 0);
int pedice=(int)inputDouble("Inserisci pedice successione: ",0);
if (a==0)
    printf("\nradice di 0=0");
else{
    printf("\nLa radice di %lf = %.16lf",a, 1.0/newtonIt(a,pedice));

    double errore=pow(10,-3);
    printf("\nCON ERRORE ASSOLUTO->La radice di %.16lf: %.14lf",a, 1.0/newtonEA(a,errore));
    //Errore Assoluto
    printf("\nCON ERRORE RELATIVO->La radice di %lf: %.16lf",a, 1.0/newtonER(a,errore));
    //Errore Relativo
}
printf("\n\n");
system("PAUSE");
}
```

Nelle ultime due funzioni sarebbe utile inserire anche un parametro passato per indirizzo per verificare il massimo valore del pedice della successione.

Calcolo approssimato del seno di un angolo espresso in radianti

La funzione seno è rappresentabile con la seguente serie di Taylor:

$$\text{sen}(x) = x - \frac{x^3}{3!} + \frac{x^5}{5!} - \frac{x^7}{7!} + \dots = \sum_{n=0}^{\infty} \left(\frac{(-1)^n x^{2n+1}}{(2n+1)!} \right)$$

È richiesto che errore assoluto e relativo siano $\leq$ dell'errore impostato o richiesto all'utente.

```
main() {  
    double errore = pow(10, -inputInt("errore=10^-"));  
    double angolo = M_PI / 4;  
    int maxn;  
    printf("\n%.16lf libreria", sin(angolo));    //con funzione della libreria math  
    printf("\n%.16lf iterativa", sinTaylorIt(angolo, 10));  
    printf("\n%.16lf ricorsiva", sinTaylorRic(angolo, 10));  
    printf("\n%.16lf errore assoluto", sinTaylorEA(angolo, errore, &maxn);  
    printf(" con massimo valore di n=%d", maxn);  
    printf("\n%.16lf errore relativo", sinTaylorER(angolo, errore, &maxn);  
    printf(" con massimo valore di n=%d", maxn);  
}
```

Metodi numerici per il CALCOLO APPROSSIMATO DELLE AREE

Problema: calcolare l'area sottesa a una curva con metodi numerici approssimati

Calcolare l'area sottesa ad una curva significa calcolare l'integrale definito; devono quindi essere soddisfatte le seguenti **ipotesi**

- la funzione deve esser limitata in un intervallo $[a;b]$
- la funzione nell'intervallo $[a;b]$ deve essere definita e continua

$$\int_a^b f(x)dx = F(b) - F(a)$$

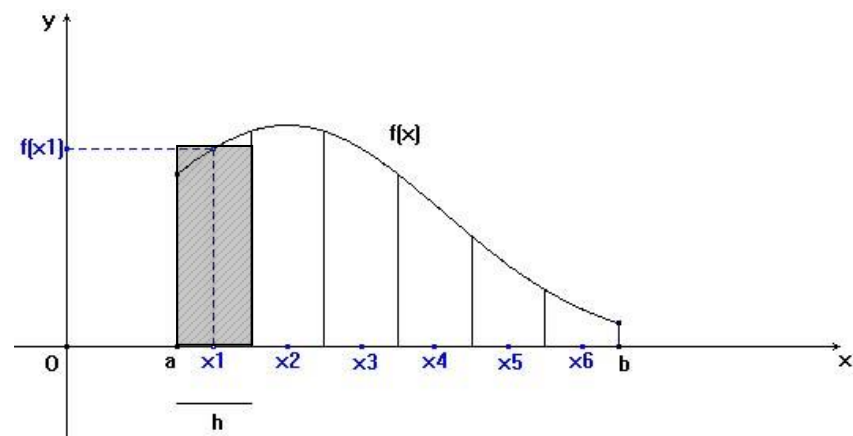
In molte funzioni, soprattutto con termini trascendenti e razionali, il calcolo di una funzione primitiva $F(x)$ può rivelarsi un'operazione complessa: l'integrazione numerica (chiamata anche quadratura) permette di ottenere un valore approssimato dell'integrale definito evitando di determinare l'espressione analitica della primitiva.

Ci sono vari metodi numerici per poter calcolare l'area sottesa ad una curva:

Metodo dei rettangoli

Questo metodo deriva direttamente dalla definizione di integrale di Riemann. Prende in considerazione n sottointervalli della stessa ampiezza uguale h e calcola la somma delle aree dei rettangoli di base h e di altezza $f(x_c)$ dove x_c è il punto centrale(medio) di ogni intervallo.

Procedura:



Si prende in considerazione l'intervallo $[a;b]$ e lo si divide in n intervalli tutti di ampiezza costante:

$$h=(b-a)/n$$

Si calcola ora il punto centrale del primo intervallo definito con la formula

$$x_1=a+(h/2)$$

L'area del primo rettangolo è quindi $A_1 = h \cdot f(x_1)$.

L'ascissa del punto medio del secondo rettangolo è $x_2 = x_1 + h$, l'area del secondo rettangolo è $A_2 = h \cdot f(x_1 + h)$,

l'ascissa del punto medio del terzo rettangolo è $x_3 = x_2 + h$, l'area del secondo rettangolo è $A_3 = h \cdot f(x_2 + h)$, l'ascissa del punto medio del quarto rettangolo è $x_4 = x_3 + h$, l'area del quarto rettangolo è $A_4 = h \cdot f(x_3 + h)$

L'area approssimata sottesa alla funzione è la sommatoria delle aree dei singoli rettangoli

$$A = h \cdot \sum_{c=1}^n f(x_c)$$

Più si aumenta il numero degli intervalli più il risultato si avvicinerà al valore effettivo dell'area.

Prendiamo in considerazione la funzione $f(x) = x^2 - \sin x$ nell'intervallo $[2;10]$; l'integrale è:

$$\int (x^2 - \sin(x)) dx = \frac{x^3}{3} + \cos(x) = 330,65 \text{ circa}$$

$$\int_2^{10} (x^2 - \sin(x)) dx = \left[\frac{x^3}{3} + \cos(x) \right]_2^{10} = 330,65208359265 \dots$$

Algoritmo:

```
double f(double x)
{
    return pow(x, 2) - sin(x);
}
double rettangoli(double a, double b, int inter)
{
    double m, area = 0, h;
    int i;
    h = (b - a) / inter;

    m = a + (h / 2); // pto medio del 1° intervallo

    for (i = 1; i <= inter; i++)
    {
        area += h * f(m);
        m += h;
    }
    return area;
}

int inputInt(char mess[], int min)
{
    {
        int n;
        do
        {
            printf("%s", mess);
            scanf("%d", &n);
        }
        while (n <= min);
        return n;
    }
}

main()
{
    double a = 2, b = 10;
    int intervalli;
    printf("\nCALCOLO APPROSSIMATO AREA SOTTESA A UNA CURVA: METODO DEI RETTANGOLI\n");
    intervalli = inputInt("\ninserisci numero di intervalli: ", 0);
    printf("\nL'area calcolata con il metodo dei rettangoli è: %.14lf", rettangoli(a, b, intervalli));
}
```

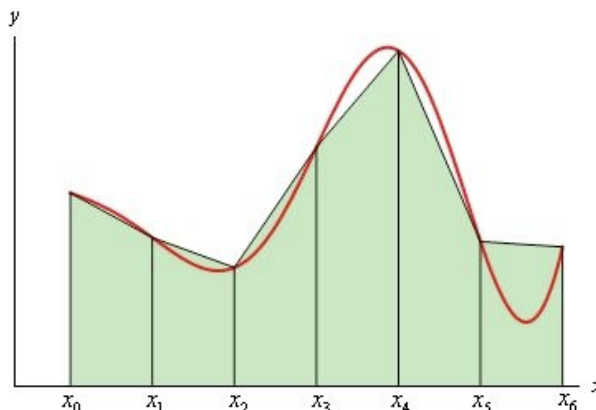

Si può ottenere un'area più precisa utilizzando il metodo dei trapezi:

Metodo dei trapezi

Come nel metodo dei rettangoli la funzione viene divisa in n sotto intervalli di ampiezza costante

$$h=(b-a)/n.$$

A differenza del metodo precedente, invece di calcolare l'area di n rettangoli, vengono calcolate le aree di n trapezi, tutti di altezza h . L'area sottesa alla funzione viene così approssimata dalla somma delle n aree aree. L'area del primo trapezio è data da:



$$\frac{(base\ maggiore + base\ minore) * altezza}{2} = \frac{(f(x_0) + f(x_1)) * h}{2}$$

In generale la prima base va a coincidere con la $f(x_i)$ dove x_i è l'estremo inferiore del sottointervallo, la seconda base va a coincidere con la $f(x_{i+1})$ dove x_{i+1} è l'estremo superiore del sottointervallo. Sommando l'area di tutti i trapezi, l'area approssimata al di sotto della curva risulta:

$$S_n \cong \frac{h}{2} * \sum_{i=1}^n (f(x_i) + f(x_{i+1})) = \frac{h}{2} * [f(a) + 2 * \sum_{i=0}^{n-1} f(x_i) + f(b)]$$

Algoritmo:

```
#include <stdio.h>
double f(double x)
{
    return pow(x, 2) - sin(x);
}
double trapezi(double a, double b, int inter)
{
    double sommabasi, area = 0, h;
    int i;
    h = (b - a) / inter;
    for (i = 1; i <= inter; i++)
    {
        sommabasi=f(a)+f(a+h);
        area += sommabasi*h/2;
        a+= h;
    }
    return area;
}
double rettangoli(double a, double b, int inter)
{
    double m, area = 0, h;
    int i;
    h = (b - a) / inter;
    m = a + (h / 2); // pto medio del 1° intervallo
    for (i = 1; i <= inter; i++)
    {
        area += h * f(m);
        m += h;
    }
    return area;
}
```

```

int inputInt(char mess[],int min)
{
    int n;
    do
    {
        printf("%s",mess);
        scanf("%d",&n);
    }
    while (n<=min);
    return n;
}

main()
{
    double a = 2, b = 10;
    int intervalli;
    printf("\nCALCOLO APPROSSIMATO AREA SOTTESA A UNA CURVA: METODO DEI TRAPEZI\n");
    intervalli = inputInt("\ninserisci numero di intervalli: ",0) ;
    printf("\nL'area calcolata con il metodo dei TRAPEZI è: %.14lf", trapezi(a, b, intervalli));
}

```

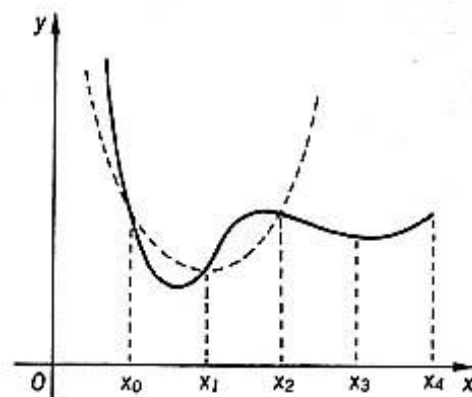
L'ultimo metodo che andremo ad analizzare è quello più utilizzato in quanto la sua precisione è superiore a quella dei metodi fin ora analizzati.

Metodo di Cavalieri-Simpson

Come nei metodi precedenti la funzione viene divisa in n sotto intervalli di ampiezza costante $h=(b-a)/n$.

Il numero di intervalli oltre a essere maggiore di 0 deve essere anche pari.

La funzione viene approssimata, in ogni sottointervallo di ampiezza $2h$, con una parabola passante per tre punti distinti; ad esempio, ragionando nel primo "doppio intervallo", con la parabola passante per x_0, x_0+h e x_0+2h ;



In generale la parabola passerà per

- Il punto $(x_i, f(x_i))$ dove x_i è l'estremo inferiore del sotto intervallo
- Il punto (punto medio dell'intervallo) è $(x_i + h, f(x_i + h))$ che rappresenta il valore che la funzione assume nel punto medio del *doppio* intervallo.
- Il punto $(x_i + 2h, f(x_i + 2h))$ dove x_{i+1} è l'estremo superiore del sottointervallo

Si può calcolare l'area sottesa ad ogni singola sezione di parabola tramite questa formula:

$$A = \frac{1}{3} * h * (f(x_i) + 4 * f(x_{i+1}) + f(x_{i+2})) = \frac{b-a}{3n} * [f(x_i) + 4 * f(x_i + h) + f(x_i + 2h)]$$

E di conseguenza si può calcolare anche l'area approssimata sottesa alla funzione, sommando le aree sottese alle $n/2$ parabole:

$$\int_a^b f(x) * dx \cong A_n = \frac{h}{3} * \sum_{i=0}^{\frac{n}{2}-1} [f(x_{i*2}) + 4 * f(x_{i*2} + h) + f(x_{i*2} + 2 * h)]$$

Algoritmo

```

double CavalieriSimpson(double a, double b, int inter)
{
    double h, area = 0;

```

```

int j;
h = (b - a) / inter;
for (j = 1; j <= inter / 2; j++) //da 1 a n/2 oppure da 0 a n/2-1
{
    area += h / 3 * (f(a) + 4 * f(a + h) + f(a + 2 * h));
    a += 2 * h;
}
return area;
}

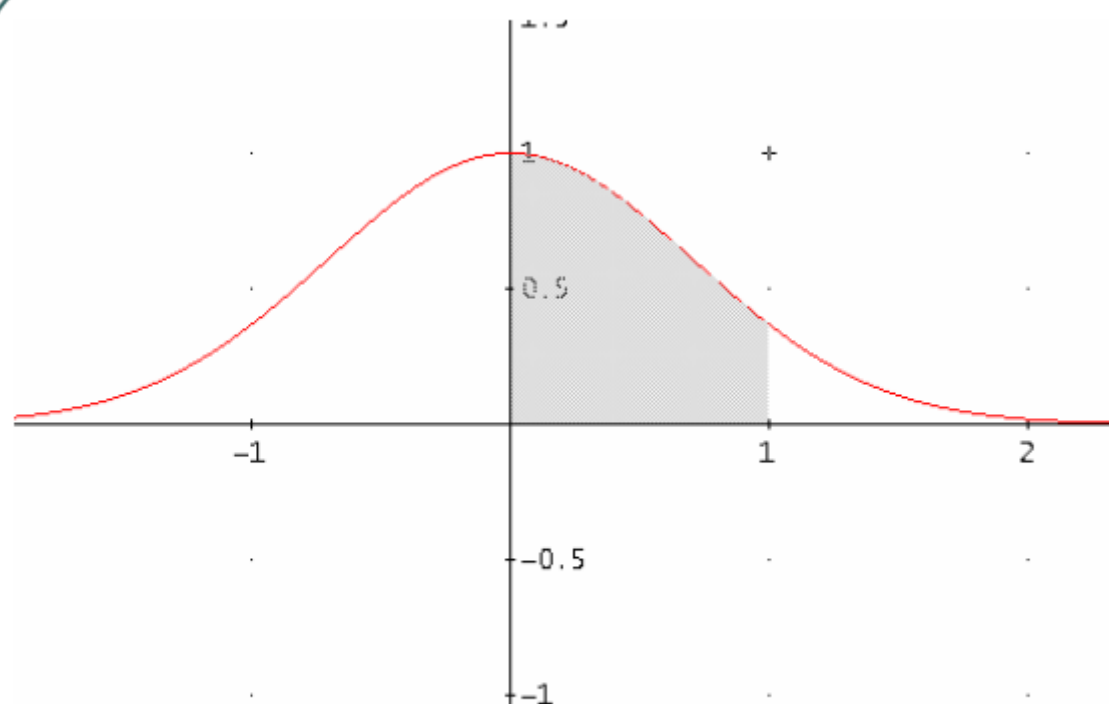
```

Un caso concreto

- Possiamo adesso utilizzare con sicurezza i nostri metodi per calcolare un integrale non risolubile con metodi classici.

$$\int_0^1 e^{-x^2} dx$$

- Questi sono i casi in cui l'integrazione numerica è veramente **utile** (indispensabile)



Risultati ottenuti

Metodo	Area	Errore
Rettangoli	1,434517	0,090609
Trapezi	1,463155	0,000503
Parabole	1,462652	0,000000

Conclusioni

- **Integrazione numerica**
 - Tecniche **approssimate** per il calcolo di integrali definiti
 - Utilizzata per calcolare gli integrali **non risolubili** con l'analisi classica
 - Ideale per il **calcolo computerizzato**
 - Facilità di traduzione in **algoritmi**
 - Facilità di adattamento per **fogli elettronici**

Conclusioni

- **Metodi di integrazione numerica**
 - **Metodo dei rettangoli**
 - **Rapidità di calcolo:** elevata
 - **Precisione:** scarsa
 - **Metodo dei trapezi**
 - **Rapidità di calcolo:** media
 - **Precisione:** media
 - **Metodo delle parabole**
 - **Rapidità di calcolo:** bassa
 - **Precisione:** elevata

Calcolo area sottesa alla funzione $y=x^3$

Confronto tra metodi numerici

```
#include <stdio.h>
#include <stdlib.h>
#include <math.h>
double f(double x)
{
    return pow(x,3);
}
double rettangoli(double sin, double des, int inte)
{
    double h=(des-sin)/inte, area=0;
    int i;
    double m=sin+h/2;
    for (i=1;i<=inte;i++)
    {
        area+=f(m)*h;
        m+=h;
    }
    return area;
}
double trapezi(double sin, double des, int inte)
{
    double h=(des-sin)/inte, area=0;
    int i;
    for (i=1;i<=inte;i++)
    {
        area+=(f(sin)+f(sin+h))*h/2;
        sin+=h;
    }
    return area;
}
double montecarlo(double sin, double des, int pti){
    int i,dentro=0;
    double x,y;
    srand(time(NULL));
    for (i=1;i<=pti;i++)
    {
        x=rand()/(double)RAND_MAX*(des-sin)+sin;
        y=rand()/(double)RAND_MAX*f(des);
        if(y<=f(x))
            dentro++;
    }
    return (des-sin)*f(des)*dentro/(float)pti;
}
double integrale(double sin , double des)
{
    return 1/4.0*(pow(des,4)-pow(sin,4));
}
double cavalieriSimpson(double sin, double des, int inte)
{
    double h=(des-sin)/inte, area=0;
```

```

    int i;
    for (i=1;i<=inte/2;i++)
    {
        area += h / 3 * (f(sin) + 4 * f(sin + h) + f(sin + 2 * h));
        sin += 2 * h;
    }
    return area;
}
main()
{
    double a=0;
    //inputDouble("inserisci estremo a: ");
    double b=3;
    //inputDouble("inserisci estremo b: ");
    printf("\nArea di x^3 tra %lf e %lf =%lf",a,b,integrale(a,b));
    int intervalli=100;
    printf("\nRettangoli: %lf", rettangoli(a,b,intervalli));
    printf("\nTrapezi: %lf", trapezi(a,b,intervalli));
    if (intervalli%2==0)
        printf("\nCavalieri-Simpson: %lf", cavalieriSimpson(a,b,intervalli));
    else
        printf("\nNon è possibile utilizzare il metodo di Cavalieri Simpson:\nil numero degli intervalli devono essere pari");
    int punti=10000;
    printf("\nMontecarlo: %lf", montecarlo(a,b,punti));
}

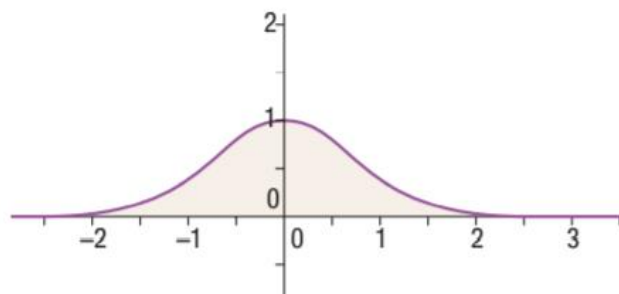
```

Esercizio del libro di testo (pag 28)

Possiamo verificare un integrale difficilmente calcolabile: l'integrale di Gauss (integrale che si presenta nello studio della densità di probabilità per variabili aleatorie continue):

$$\int_{-\infty}^{+\infty} e^{-x^2} dx = \sqrt{\pi}$$

La curva è la seguente:



```
#include <stdio.h>
```

```

#include <math.h>
#include <stddef.h>
#include <stdlib.h>
double f(double x)
{
    return pow(M_E,-pow(x,2));
}
double Montecarlo(double a, double b, int totpunti)
{
    double x,y;
    int i, sotto=0;
    for (i=1;i<=totpunti;i++)
    {
        x=(double)rand()/RAND_MAX*(b-a)+a;
        y=(double)rand()/RAND_MAX;
        if(y<=f(x))
            sotto++;
    }
    return(b-a)*f(a)*sotto/(double)totpunti;
}
int inputInt(char s[], int min)
{
    int v;
    do
    {
        printf("%s",s);
        scanf("%d", &v);
    }
    while (v<min);
    return v;
}
double inputDouble(char s[], double min)
{
    double v;
    do
    {
        printf("%s",s);
        scanf("%lf", &v);
    }
    while (v<min);
    return v;
}
double rettangoli(double a, double b, int inter)

```



```

{
    double m, area = 0, h;
    int i;
    h = (b - a) / inter;
    m = a + (h / 2); // pto medio del 1° intervallo
    for (i = 1; i <= inter; i++)
    {
        area += h * f(m);
        m += h;
    }
    return area;
}

main()
{
    srand(time(NULL));
    double a=inputDouble("\n1^ estremo di integrazione: ", 0),b=inputDouble("\n2^
estremo di integrazione: ",a);
    printf("\nCALCOLO DELL'AREA SOTTESA ALLA GAUSSIANA TRA -%lf e %lf",b,b);
    double valoreMontecarlo=Montecarlo(a,b,inputInt("\n\nQuanti punti: ",1));
    printf("\n-con metodo MONTECARLO: %lf",2*valoreMontecarlo);
    //double valoreVero=sqrt(M_PI);
    //printf("\n\n\"valore vero\":%lf", valoreVero);
    //printf("\nERRORE ASSOLUTO: %lf",fabs(valoreVero-2*valoreMontecarlo));

    double valoreRettangoli=rettangoli(a,b,inputInt("\n\nQuanti intervalli: ",1));
    printf("\n-con metodo dei RETTANGOLI: %lf",2*valoreRettangoli);
    printf("\n\n");
    system("PAUSE");
}

```

Esercizio

Data la funzione $y = \frac{1}{1+\sqrt{x}}$ calcolare la misura dell'area della parte di piano delimitata dal grafico della funzione, dall'asse x e dalle rette $x=0$ e $x=\alpha$.

$$A = \int_0^{\alpha} \frac{1}{1+\sqrt{x}} dx = 2 * \ln \frac{e^{\sqrt{\alpha}}}{1+\sqrt{\alpha}}$$

Con $\alpha=16$ $A=4,7811241751318$

```

#include <stdio.h>
#include <stdlib.h>
#include <math.h>
double f(double x)
{
    return 1/(1+sqrt(x));
}
double CavalieriSimpson(double a, double b, int inter)
{
    double h, area = 0;
    int j;
    h = (b - a) / inter;
    for (j = 1; j <= inter / 2; j++) //da 1 a n/2 oppure da 0 a n/2-1
    {
        area += h / 3 * (f(a) + 4 * f(a + h) + f(a + 2 * h));
        a += 2 * h;
    }
    return area;
}

```

```

double trapezi(double a, double b, int inter)

```

```

{
    double sommabasi, area = 0, h;
    int i;
    h = (b - a) / inter;
    for (i = 1; i <= inter; i++)
    {
        sommabasi=f(a)+f(a+h);
        area += sommabasi*h/2;
        a+= h;
    }
    return area;
}

```

```

double rettangoli(double a, double b, int inter)

```

```

{
    double m, area = 0, h;
    int i;
    h = (b - a) / inter;
    m = a + (h / 2); // pto medio del 1° intervallo
    for (i = 1; i <= inter; i++)
    {
        area += h * f(m);
        m += h;
    }
    return area;
}

```

```
int inputInt(char mess[],int min)
```

```
{
    int n;
    do
    {
        printf("%s",mess);
        scanf("%d",&n);
    }
    while (n<=min);
    return n;
}
```

```
main()
```

```
{
    double a = 0, b = 16;
    double espo=sqrt(b);
    printf("\nCALCOLO con integrale definito: %lf\n",2*log(pow(M_E,espo)/(1+espo)));
    int intervalli;
    printf("\nCALCOLO APPROSSIMATO AREA SOTTESA A UNA CURVA: METODO DEI TRAPEZI\n");
    intervalli = inputInt("\ninserisci numero di intervalli: ",0) ;
    printf("\nL'area calcolata con il metodo dei TRAPEZI e': %.14lf", trapezi(a, b, intervalli));
}
```